Nama : Mohamad Zaky Bahtiar Arifianto
Matkul : Praktikum Pemrogramman Berorientasi Obyek
Jurusan : Teknik Informatika
NRP : 3125522015

Tugas Pertemuan 11

BAB I: TUJUAN PRAKTIKUM & DASAR TEORI

Praktikum Pemrograman Berorientasi Obyek Modul 11 menitikberatkan pada perombakan arsitektur penyimpanan data statis menjadi dinamis menggunakan Java Collections Framework. Studi kasus yang dikembangkan adalah Sistem Manajemen Program Makan Bergizi Gratis (MBG) berbasis desktop JavaFX.

1.1 Tujuan Praktikum

- **Penyimpanan Objek Dinamis:** Mampu menyimpan dan mengorganisasi kumpulan objek domain (Menu, Pengiriman, BahanBaku, Sekolah) secara dinamis menggunakan ArrayList.
- **Implementasi Generics Type-Safe:** Menerapkan generics tipe terdefinisi seperti List<Menu> dan List<Pengiriman> untuk menjamin keamanan tipe (compile-time type safety) dan menghilangkan casting eksplisit.
- **Operasi Koleksi Lengkap (CRUD):** Merancang method tambah, cari, tampilkan, dan hapus yang terisolasi di dalam class pengelola.
- **Validasi Keunikan & Integritas Data:** Menjaga agar kode unik (misal: kodeMenu, idPengiriman) tidak dapat diduplikasi dan data tidak valid/null langsung ditolak.
- **Enkapsulasi Ketat:** Mengunci referensi list dengan private final serta mencegah manipulasi ilegal dari class eksternal dengan membungkus koleksi menggunakan Collections.unmodifiableList().

1.2 Dasar Teori Collections Framework & ArrayList

Pada bahasa Java, array konvensional (misal Menu[] daftar) memiliki kelemahan mendasar yaitu ukurannya bersifat tetap (fixed size) sejak pertama kali diinstansiasi. Ketika jumlah entitas pangan atau sekolah bertambah melebihi kapasitas awal array, sistem harus membuat array baru dan menyalin seluruh elemen secara manual. Sebaliknya, ArrayList merupakan implementasi antarmuka List yang ukurannya dapat bertumbuh secara elastis seiring penambahan objek baru.

Struktur deklarasi standar pada pemrograman berorientasi obyek yang baik adalah mengacu pada interface (coding to interface):

```
List<Menu> daftarMenu = new ArrayList<>();
```

List adalah interface yang mendefinisikan kontrak operasi koleksi berurutan, sedangkan ArrayList adalah kelas konkret yang mengimplementasikan struktur data berbasis resizable-array. Parameter generics <Menu> mengunci koleksi agar hanya dapat menampung objek turunan Menu, mencegah runtime ClassCastException.

BAB II: ARSITEKTUR MODEL & PENGELOLA KOLEKSI PROYEK MBG

Dalam perancangan sistem MBG Modul 11, diterapkan pemisahan tanggung jawab (Separation of Concerns) yang jelas antara Objek Domain tunggal dan Class Manajer Pengelola Koleksi.

2.1 Objek Domain (Menu & Pengiriman) vs Class Pengelola (PengelolaMenu & PengelolaDistribusi)

- **Objek Domain Menu:** Bertanggung jawab atas integritas data satu unit hidangan gizi. Objek ini mengelola atribut kodeMenu, namaMenu, kaloriPerPorsi, proteinGram, karbohidratGram, biayaPerPorsi, dan relasi agregasi kebutuhan bahan baku per 100 porsi. Kelas Menu memvalidasi nilai input tidak boleh kosong dan angka bernilai positif.
- **Class Pengelola PengelolaMenu:** Bertanggung jawab atas pengelolaan kumpulan banyak objek Menu. Kelas ini mengontrol penambahan menu baru, pencarian cepat berdasarkan kode unik, penghapusan, dan visualisasi koleksi. Koleksi internal disimpan secara privat sehingga Main atau Controller tidak dapat memodifikasi list secara langsung.

2.2 Mekanisme Enkapsulasi Ketat & Pencegahan Modifikasi Liar

Prinsip Information Hiding menuntut agar struktur data internal tidak boleh diekspos ke publik. Pada PengelolaMenu, koleksi dideklarasikan sebagai:

```
private final List<Menu> daftarMenu = new ArrayList<>();
```

Kata kunci final menjamin referensi daftarMenu tidak dapat diganti dengan instance list lain. Untuk mencegah class lain memanggil method mutable seperti daftarMenu.add() atau daftarMenu.clear(), getter koleksi dibungkus secara read-only:

```
public List<Menu> getDaftarMenu() {
    return Collections.unmodifiableList(daftarMenu);
}
```

Jika ada kode luar yang mencoba memanipulasi list ini, Java Virtual Machine akan langsung melemparkan UnsupportedOperationException saat runtime, menjaga koleksi tetap aman dan konsisten.

2.3 Aturan Bisnis & Logika Penolakan Duplikasi (Fail-Fast)

Sebelum objek baru dimasukkan ke dalam koleksi, PengelolaMenu memeriksa validitas parameter dan keunikan identitas:

```
public void tambahMenu(Menu menu) {
    if (menu == null) {
        throw new IllegalArgumentException("Objek menu tidak boleh null.");
    }
    if (cariMenu(menu.getKodeMenu()) != null) {
        throw new IllegalArgumentException("Kode menu '" + menu.getKodeMenu() + "' sudah
terdaftar.");
    }
    daftarMenu.add(menu);
}
```

Pengecekan keunikan dilakukan sebelum operasi add() dieksekusi. Hal ini menjamin bahwa penolakan data tidak mengubah ukuran atau isi koleksi (state immutability on failure).

BAB III: IMPLEMENTASI KODE PROGRAM UTAMA

3.1 Kode Objek Domain: Menu.java

```
package P11_3125522015_MohamadZaky.model;

import java.util.Collections;
import java.util.LinkedHashMap;
import java.util.Map;

public class Menu {
    private String kodeMenu;
    private String namaMenu;
    private double kaloriPerPorsi;
    private double proteinGram;
    private double karbohidratGram;
    private double biayaPerPorsi;
    private final Map<BahanBaku, Integer> komposisiPer100Porsi;

    public Menu(String kodeMenu, String namaMenu, double kaloriPerPorsi,
                double proteinGram, double karbohidratGram, double biayaPerPorsi) {
        setKodeMenu(kodeMenu);
        setNamaMenu(namaMenu);
        setKaloriPerPorsi(kaloriPerPorsi);
        setProteinGram(proteinGram);
        setKarbohidratGram(karbohidratGram);
        setBiayaPerPorsi(biayaPerPorsi);
        this.komposisiPer100Porsi = new LinkedHashMap<>();
    }

    public void tambahBahanBaku(BahanBaku bahan, int kebutuhanPer100Porsi) {
        if (bahan == null) throw new IllegalArgumentException("Objek BahanBaku tidak boleh
null.");
        if (kebutuhanPer100Porsi <= 0) throw new IllegalArgumentException("Kebutuhan bahan harus
> 0.");
        komposisiPer100Porsi.put(bahan, kebutuhanPer100Porsi);
    }

    public final void setKodeMenu(String kodeMenu) {
        if (kodeMenu == null || kodeMenu.trim().isEmpty()) {
            throw new IllegalArgumentException("Kode menu tidak boleh kosong.");
        }
        this.kodeMenu = kodeMenu.trim().toUpperCase();
    }

    public final void setNamaMenu(String namaMenu) {
        if (namaMenu == null || namaMenu.trim().isEmpty()) {
            throw new IllegalArgumentException("Nama menu tidak boleh kosong.");
        }
        this.namaMenu = namaMenu.trim();
    }

    public String getKodeMenu() { return kodeMenu; }
    public String getNamaMenu() { return namaMenu; }
    public double getKaloriPerPorsi() { return kaloriPerPorsi; }
    public double getProteinGram() { return proteinGram; }
    public double getKarbohidratGram() { return karbohidratGram; }
    public double getBiayaPerPorsi() { return biayaPerPorsi; }
    public Map<BahanBaku, Integer> getKomposisiPer100Porsi() {
        return Collections.unmodifiableMap(komposisiPer100Porsi);
```

```
    }
}
```

3.2 Kode Class Pengelola: PengelolaMenu.java

```
package P11_3125522015_MohamadZaky.model;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class PengelolaMenu {
    private final List<Menu> daftarMenu = new ArrayList<>();

    public void tambahMenu(Menu menu) {
        if (menu == null) {
            throw new IllegalArgumentException("Objek menu tidak boleh null.");
        }
        if (cariMenu(menu.getKodeMenu()) != null) {
            throw new IllegalArgumentException("Kode menu '" + menu.getKodeMenu() + "' sudah
terdaftar.");
        }
        daftarMenu.add(menu);
    }

    public Menu cariMenu(String kodeMenu) {
        if (kodeMenu == null || kodeMenu.trim().isEmpty()) return null;
        for (Menu menu : daftarMenu) {
            if (menu.getKodeMenu().equalsIgnoreCase(kodeMenu.trim())) {
                return menu;
            }
        }
        return null;
    }

    public boolean hapusMenu(String kodeMenu) {
        Menu menu = cariMenu(kodeMenu);
        if (menu == null) return false;
        return daftarMenu.remove(menu);
    }

    public void tampilkanMenu() {
        if (daftarMenu.isEmpty()) {
            System.out.println("Koleksi menu MBG masih kosong.");
            return;
        }
        System.out.println("=== DAFTAR MENU GIZI MBG ===");
        for (Menu menu : daftarMenu) {
            System.out.println(menu.getKodeMenu() + " - " + menu.getNamaMenu()
                    + " | Kalori: " + menu.getKaloriPerPorsi() + " kkal"
                    + " | Biaya: Rp " + (long) menu.getBiayaPerPorsi());
        }
        System.out.println("Total menu terdaftar: " + daftarMenu.size());
    }

    public int getJumlahMenu() { return daftarMenu.size(); }
    public List<Menu> getDaftarMenu() { return Collections.unmodifiableList(daftarMenu); }
    public void kosongkan() { daftarMenu.clear(); }
```

```
}
```

BAB IV: TABEL SKENARIO PENGUJIAN & LOG EKSEKUSI PROGRAM

Pengujian dilakukan secara berurutan pada satu objek PengelolaMenu untuk menguji seluruh aspek CRUD, validasi masukan tidak valid, penolakan kode duplikat, dan enkapsulasi koleksi.

No	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual	Data Akhir
1	Tampilkan koleksi saat kondisi awal kosong	Menampilkan pesan koleksi kosong tanpa error	Pesan: 'Koleksi menu MBG masih kosong.'	**0**
2	Tambah Menu valid: MNU-01 dan MNU-02	Kedua menu tersimpan ke dalam List<Menu>	Kedua menu tersimpan, size = 2	**2**
3	Cari Menu berdasarkan kode valid ('MNU-02')	Mengembalikan objek Menu 'Paket B'	Ditemukan: MNU-02 - Paket B	**2**
4	Tambah Menu dengan kode duplikat ('MNU-01')	Operasi add ditolak, melempar IllegalArgumentException	Ditolak: Kode menu 'MNU-01' sudah terdaftar	**2**
5	Tambah Menu dengan objek null	Operasi add ditolak, melempar IllegalArgumentException	Ditolak: Objek menu tidak boleh null	**2**
6	Tambah Menu atribut tidak valid (kode kosong)	Konstruktor menolak, melempar IllegalArgumentException	Ditolak: Kode menu tidak boleh kosong	**2**
7	Hapus Menu terdaftar ('MNU-01')	Method hapusMenu menghasilkan true, elemen dihapus	Mengembalikan true, data berkurang	**1**
8	Cari Menu yang telah dihapus ('MNU-01')	Method cariMenu mengembalikan null	Hasil: null (Menu tidak ditemukan)	**1**
9	Hapus Menu tidak terdaftar ('MNU-999')	Method hapusMenu menghasilkan false secara aman	Mengembalikan false tanpa runtime crash	**1**
10	Tampilkan seluruh isi koleksi akhir	Menampilkan 1 data tersisa beserta total elemen	Tercetak rincian MNU-02, Total menu: 1	**1**
11	Uji Enkapsulasi: getDaftarMenu().add(...)	Ditolak oleh UnsupportedOperationException (read-only)	Melempar UnsupportedOperationException	**1**



Gambar 4.1: Log Eksekusi Verifikasi Otomatis Modul 11 pada Terminal PowerShell

BAB V: JAWABAN EVALUASI RESMI MODUL 11

Pertanyaan 1: Apa perbedaan objek domain (misal: Menu) dan List<Menu>?

Objek domain Menu adalah representasi satu unit data tunggal yang menyimpan detail atribut hidangan makanan bergizi (seperti kodeMenu, namaMenu, kaloriPerPorsi, proteinGram, dan komposisi bahan baku) serta bertanggung jawab menjaga integritas state dan validasi bisnis internal dirinya sendiri. Sedangkan List<Menu> adalah struktur data koleksi (container dinamis) yang bertugas menampung,

menyusun, dan mengelola kumpulan referensi banyak objek Menu di memori. List<Menu> tidak merepresentasikan entitas makanan, melainkan wadah yang menyediakan method manipulasi kumpulan seperti penambahan, pencarian, penghapusan, dan kalkulasi jumlah elemen.

Pertanyaan 2: Mengapa indeks list kurang tepat dijadikan identitas tetap sebuah data MBG?

Indeks pada List bersifat dinamis dan relatif (volatile). Ketika sebuah elemen di awal atau tengah koleksi dihapus (misal menu pada indeks ke-0 dihapus via remove(0)), seluruh elemen di belakangnya akan otomatis bergeser maju mengisi posisi yang kosong, sehingga elemen yang semula berada pada indeks 1 berubah menjadi indeks 0. Jika sistem distribusi MBG mengacu pada nomor indeks (misalnya Surat Jalan Pengiriman merujuk 'Menu Indeks 1'), penghapusan salah satu menu akan menyebabkan menu makanan yang dikirim ke sekolah sasaran tertukar secara fatal. Oleh sebab itu, identitas entitas MBG wajib menggunakan natural unique key yang permanen seperti kodeMenu ('MNU-01') atau idPengiriman ('DST-01').

Pertanyaan 3: Mengapa pengecekan kode duplikat dilakukan sebelum operasi add()?

Secara bawaan, koleksi ArrayList mengizinkan elemen duplikat. Jika pengecekan keunikan tidak dilakukan sebelum pemanggilan add(), sistem akan menerima dua objek berbeda dengan kode ID yang sama ke dalam memori. Melakukan iterasi atau memanggil method cariMenu() sebelum operasi add() menerapkan prinsip Fail-Fast dan State Invariance: data yang melanggar aturan bisnis ditolak seketika pada gerbang awal dengan melempar IllegalArgumentException, sehingga status koleksi internal tetap bersih, jumlah data tidak bertambah sembarangan, dan integritas basis data tetap terjamin.

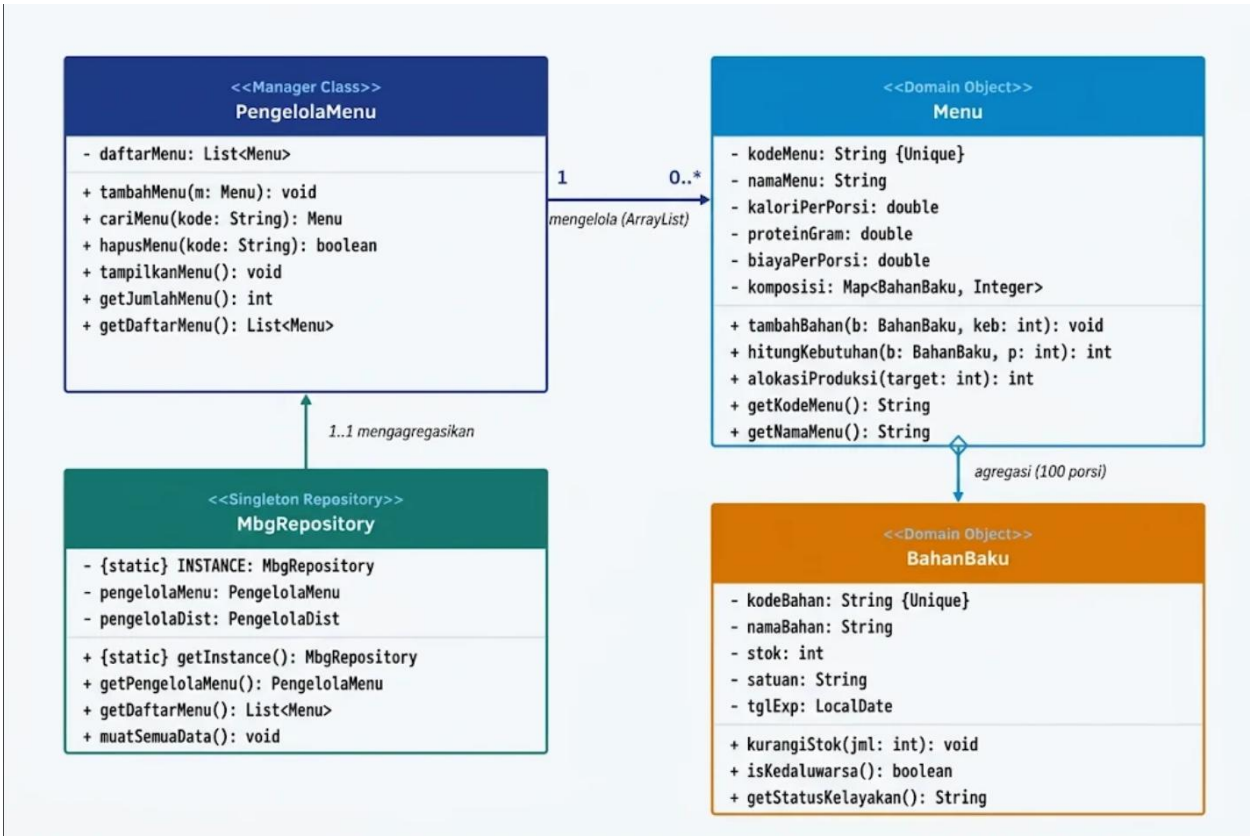
Pertanyaan 4: Apa risiko jika collection internal dapat diubah langsung oleh class lain?

Risiko utamanya adalah terjadinya kebocoran enkapsulasi (encapsulation breach). Jika referensi mutable list internal diberikan langsung ke class luar tanpa proteksi:

- **1. Bypass Validasi:**Class luar dapat memanggil method add(), remove(), atau clear() secara bebas tanpa melewati logika validasi class pengelola, sehingga data duplikat atau objek null dapat menyusup ke koleksi.
- **2. Data Inconsistency:**Terjadi ketidaksinkronan data antara memori aplikasi dengan tabel MySQL, karena class lain memodifikasi koleksi tanpa memicu query pembaruan database.
- **3. Solusi Pertahanan:**Kerusakan data ini diatasi dengan membungkus getter koleksi menggunakan Collections.unmodifiableList(daftarMenu) sehingga hanya dapat dibaca (read-only).

BAB VI: PEMBARUAN UML CLASS DIAGRAM

Berikut adalah pembaruan struktur diagram kelas yang memperlihatkan relasi One-to-Many (1..*) antara class pengelola koleksi dengan objek-objek domain MBG.



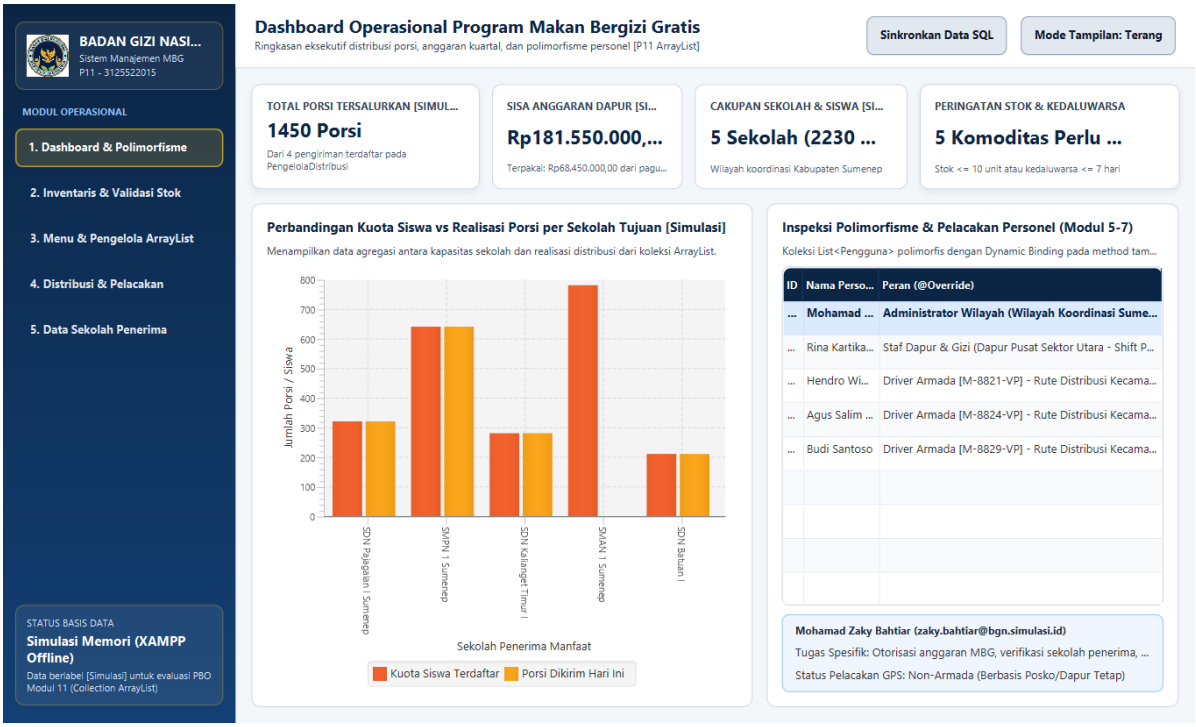
Gambar 6.1: Diagram Kelas UML Relasi 1-to-Many Antara Class Pengelola dan Objek Domain MBG

Penjelasan Relasi pada Diagram UML:

- **1. PengelolaMenu ke Menu (1-to-Many):**Satu instance PengelolaMenu menguasai dan mengelola nol hingga banyak objek Menu (1 ke 0..*) menggunakan koleksi internal ArrayList. Seluruh manipulasi data diisolasi melalui method resmi tambahMenu, cariMenu, dan hapusMenu.
- **2. PengelolaDistribusi ke Pengiriman (1-to-Many):**Satu instance PengelolaDistribusi mengelola seluruh jadwal dan armada pengiriman makanan bergizi ke sekolah tujuan.
- **3. Pengiriman ke DetailPengiriman (Composition):**Setiap Pengiriman menguasai siklus hidup satu atau lebih objek DetailPengiriman (kemasan batch termal food-grade).
- **4. Menu ke BahanBaku (Aggregation):**Objek Menu mengagregasikan objek BahanBaku dalam takaran per 100 porsi tanpa menguasai siklus hidup BahanBaku di gudang umum.

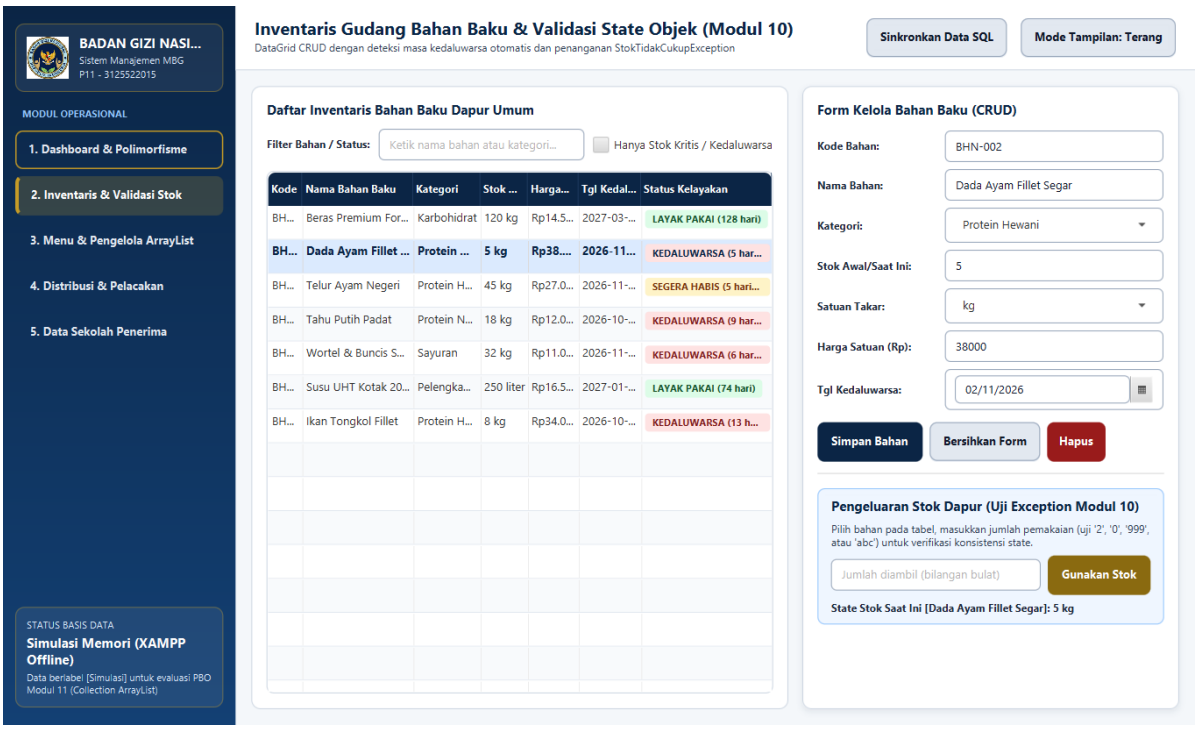
BAB VII: DOKUMENTASI TAMPILAN ANTARMUKA APLIKASI DESKTOP JAVAFX

Berikut adalah dokumentasi hasil tangkapan layar antarmuka pengguna aplikasi desktop MBG yang telah diintegrasikan dengan ArrayList dan Pengelola Koleksi Modul 11:



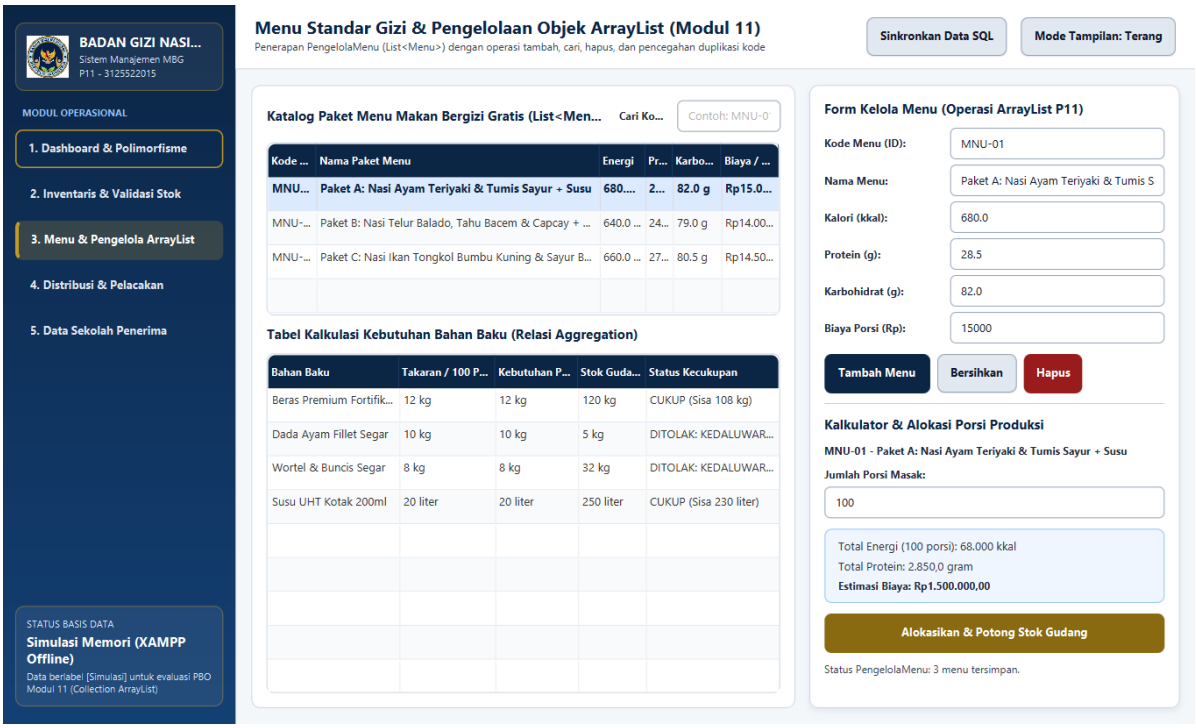
Gambar 7.1: Dashboard Operasional MBG, Ringkasan Porsi Terdistribusi & Polimorfisme Personel

Gambar 7.1 menampilkan metrik eksekutif serapan anggaran, kuota siswa penerima, grafik batang perbandingan alokasi porsi harian vs kapasitas siswa sekolah, serta tabel polimorfisme personel MBG berbasis List<Pengguna>.



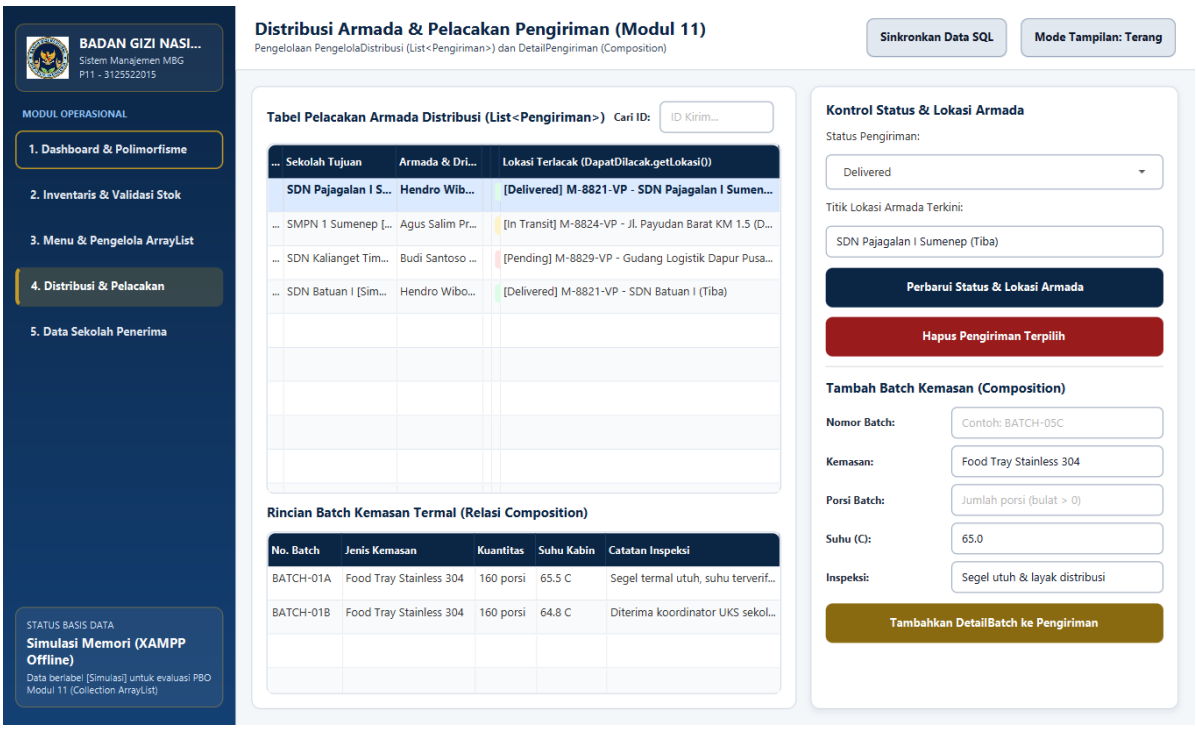
Gambar 7.2: Modul Inventaris Gudang Bahan Baku, Validasi Stok & Penanganan Exception

Gambar 7.2 mendokumentasikan DataGrid komoditas pangan dapur umum, deteksi otomatis masa kedaluwarsa bahan, dan pengujian pengurangan stok dengan try-catch-finally.



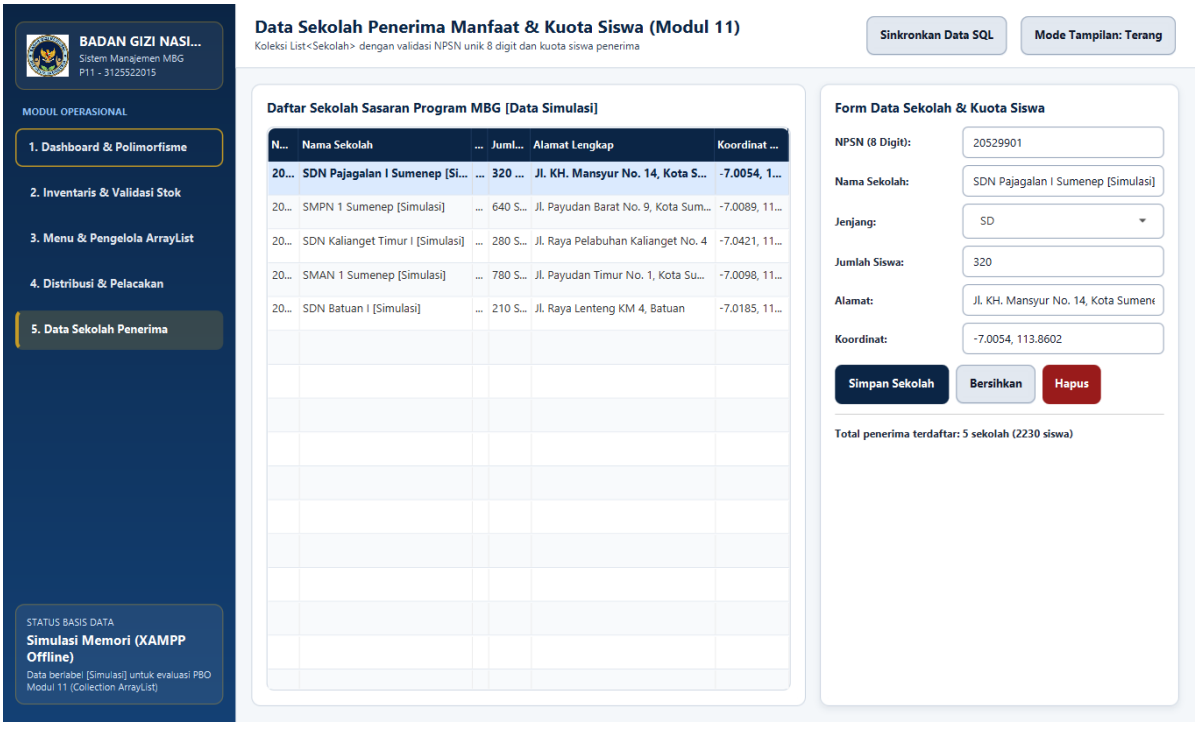
Gambar 7.3: Modul Katalog Menu Gizi, Form CRUD ArrayList PengelolaMenu & Kalkulator Nutrisi

Gambar 7.3 memperlihatkan antarmuka utama Modul 11: tabel List<Menu> terdaftar, fitur pencarian kodeMenu, form CRUD penambahan menu baru dengan validasi pencegahan duplikasi kode, serta kalkulator gizi otomatis.



Gambar 7.4: Modul Pelacakan Armada Distribusi & Komposisi Batch Kemasan Termal

Gambar 7.4 menampilkan tabel List<Pengiriman> yang terintegrasi dengan antarmuka DapatDilacak serta tabel detail batch kemasan termal (relasi composition).



Gambar 7.5: Modul Data Sekolah Sasaran Penerima Manfaat & Kuota Siswa

Gambar 7.5 mendokumentasikan pengelolaan data sekolah sasaran MBG berbasis List<Sekolah> dengan validasi NPSN 8 digit unik dan kuota siswa penerima.

BAB VIII: KESIMPULAN

Berdasarkan implementasi, refactoring, dan serangkaian pengujian yang telah dilakukan pada Praktikum Modul 11, dapat disimpulkan beberapa poin penting:

- **1. Fleksibilitas Struktur Data:**Penerapan ArrayList menggantikan array statis secara signifikan meningkatkan fleksibilitas dan skalabilitas sistem MBG dalam mengelola data pangan dan logistik yang berkembang dinamis.
- **2. Type-Safety:**Penggunaan Generics (List<Menu>, List<Pengiriman>) menjamin keamanan tipe data saat waktu kompilasi, mencegah timbulnya ClassCastException saat runtime.
- **3. Stabilitas Identitas Objek:**Penggunaan natural unique key (kodeMenu, idPengiriman) berhasil mengatasi kelemahan indeks list yang rentan bergeser akibat operasi penghapusan elemen.
- **4. Integritas Enkapsulasi:**Pemisahan class pengelola (PengelolaMenu) dan pembungkusan via Collections.unmodifiableList() sukses menegakkan prinsip enkapsulasi ketat, mencegah intervensi mutasi liar dari luar sistem.
- **5. Keandalan Sistem:**Pemeriksaan kode duplikat dan objek null sebelum operasi add() terbukti efektif menjaga kestabilan state koleksi dari data anomali atau korup.